

미래형자동차 핵심기술 전문인력양성사업

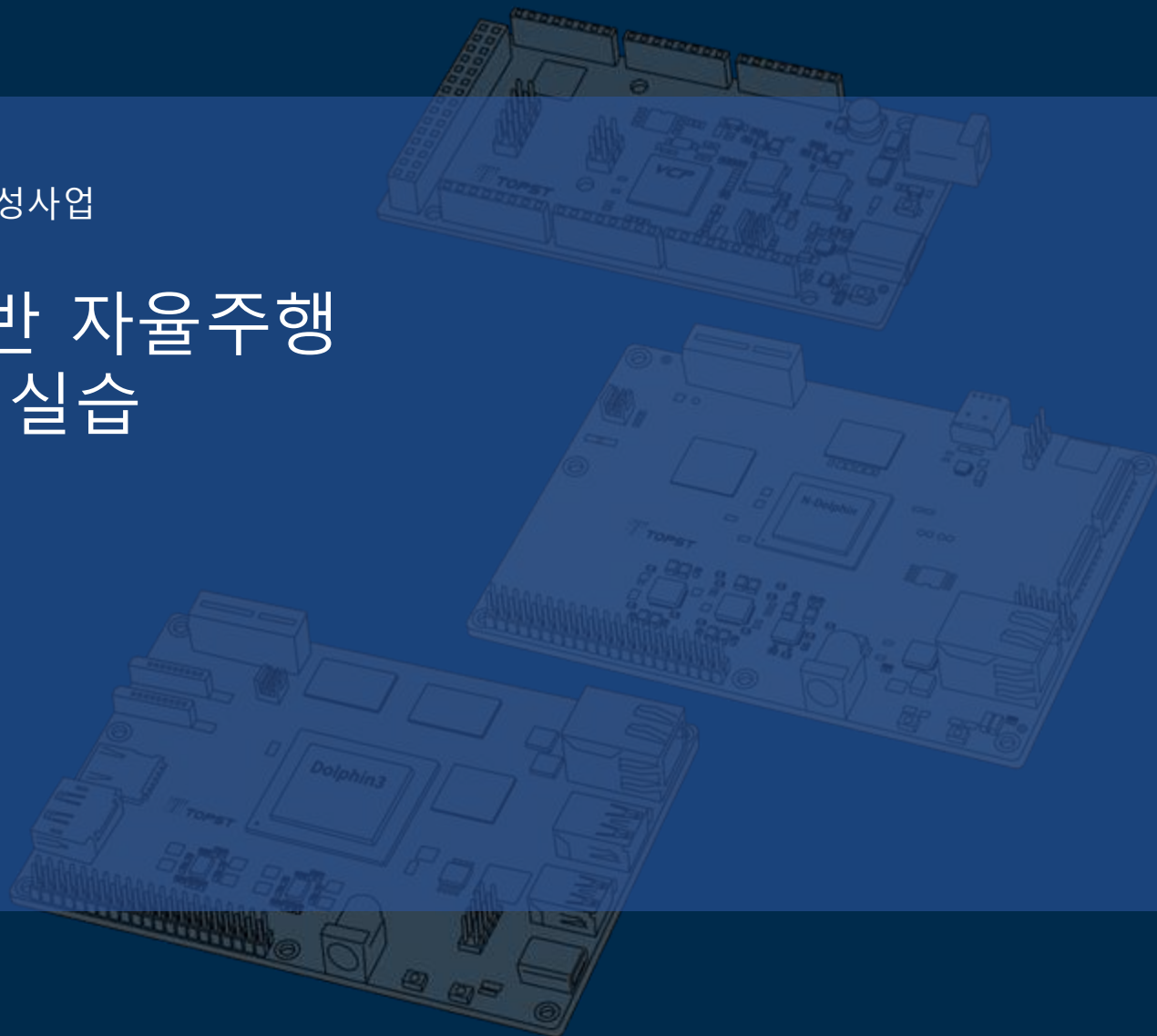
SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



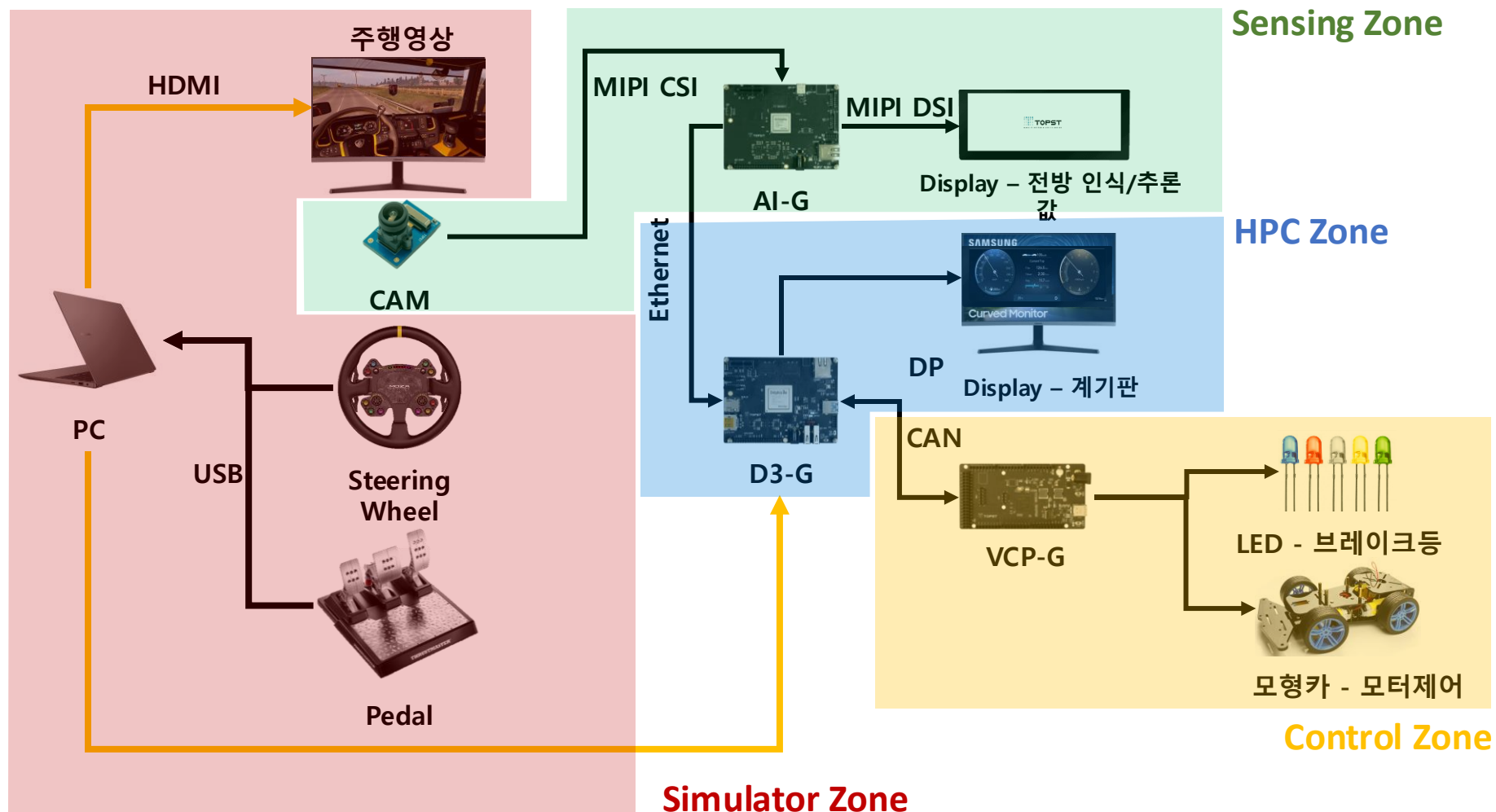
3일차

03 Simulator Zone

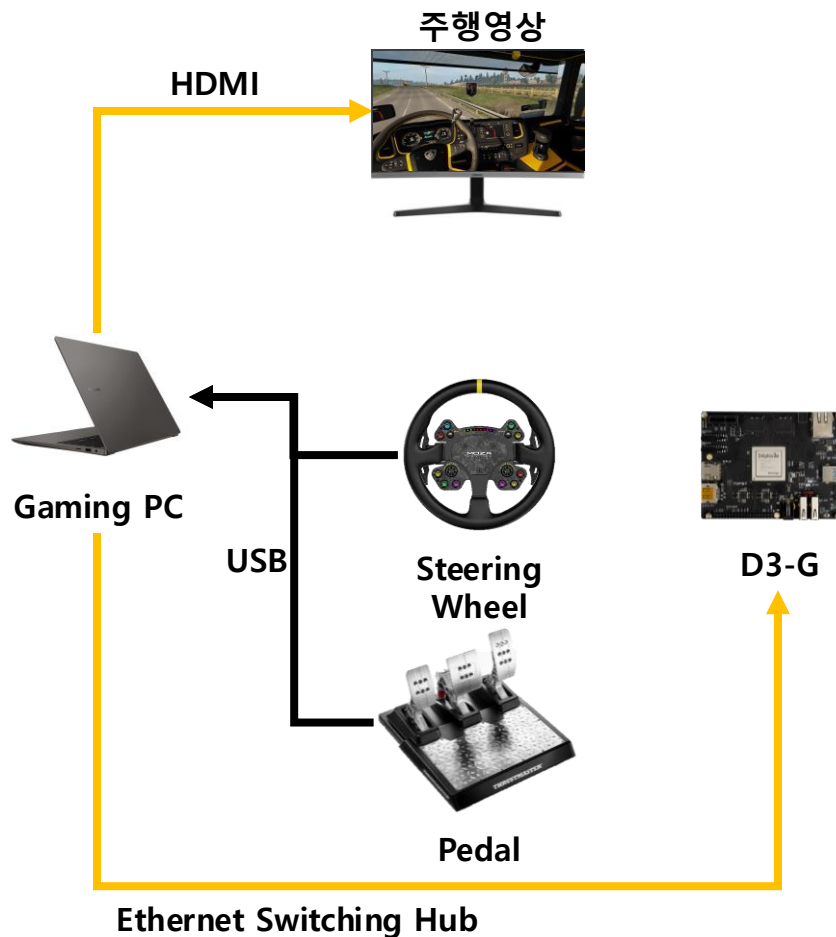
Telechips  **TOPST**



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



Simulator Zone 구현



Zone Description

'유로트럭' 시뮬레이터 내의 차량 데이터를 D3-G와 통신하기

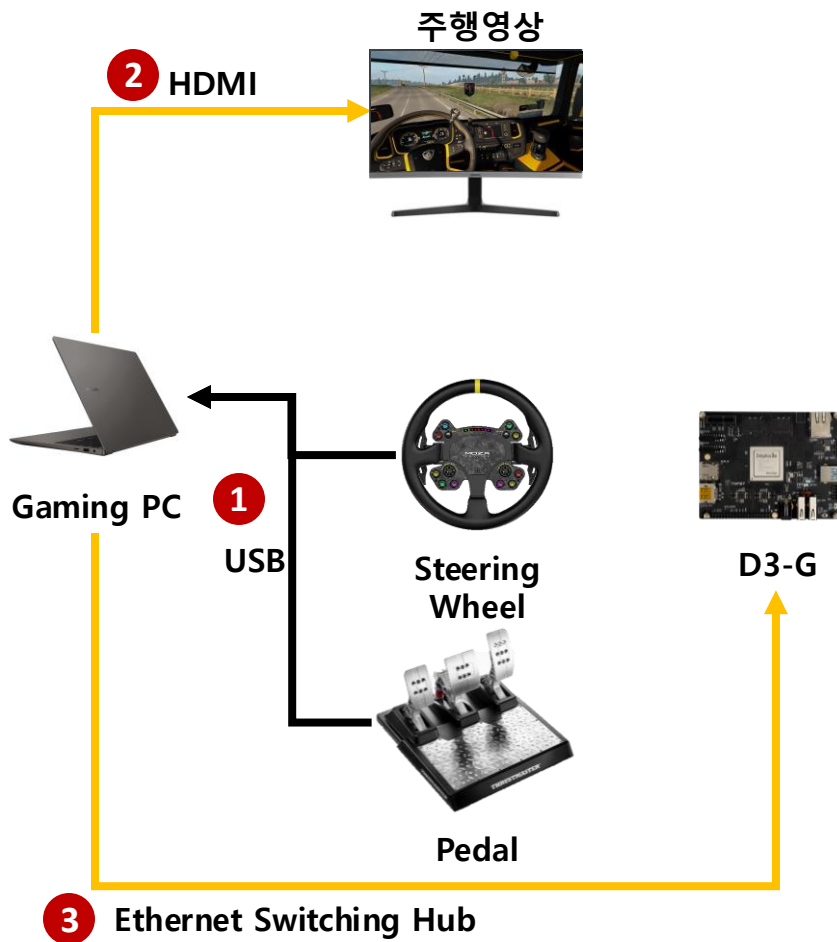
Task Goal

1. Wheel & Pedal 장비 연결
2. 시뮬레이터 설정
3. 시뮬레이터와 PC 통신
4. PC와 D3-G 데이터 송수신

Simulator Guide

- 제공된 Gaming PC 환경에서 유로트럭 실행(팀당 1대)
 - 준비된 Steam 계정의 유로트럭 실행
- 실험을 위한 유로트럭 시뮬레이션의 사용기능은 아래와 같이 안내
- 차량 조종은 제공된 steering wheel 과 pedal로 진행
- 차량에 있는 라이트 제어키는 키보드로 진행
 - 방향 지시등 (왼쪽/오른쪽)
 - 비상등
 - 전방 라이트

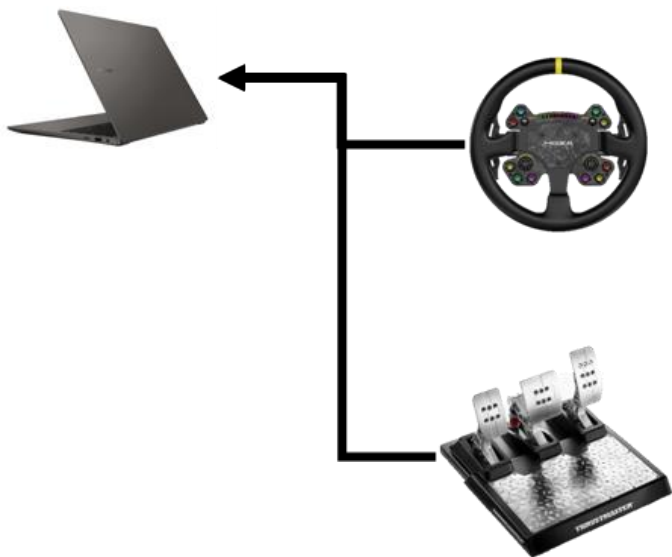
Simulator Zone 장치간 연결



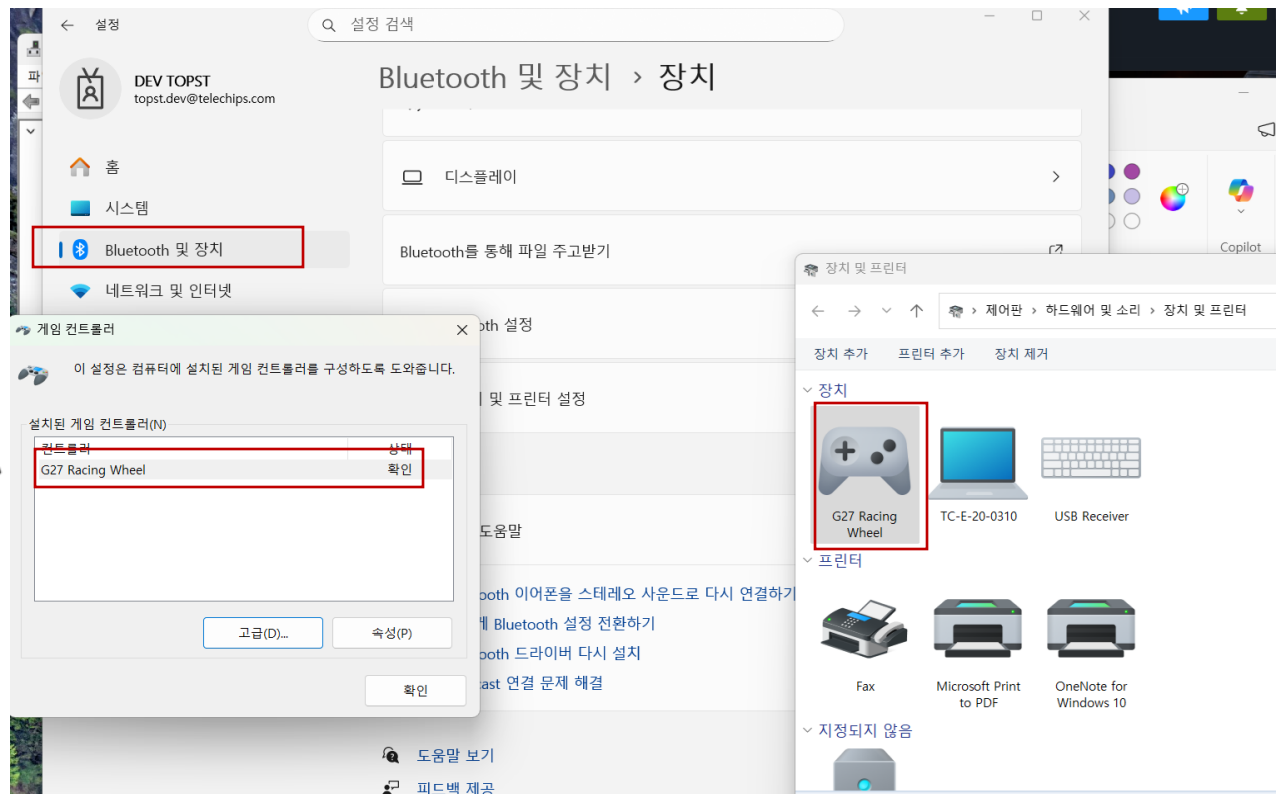
장치간 연결

1. Steering Wheel과 Pedal을 Gaming PC에 연결 (시뮬레이터 연동)
2. Gaming PC와 모니터 HDMI 연결
3. D3-G 보드와 게이밍 PC와의 이더넷 연결(제공된 허브)

USB Wheel 장비 연결



Wheel 와 Pedal의
USB A type 으로 PC 연결 시, 바로 인식



Simulator Zone

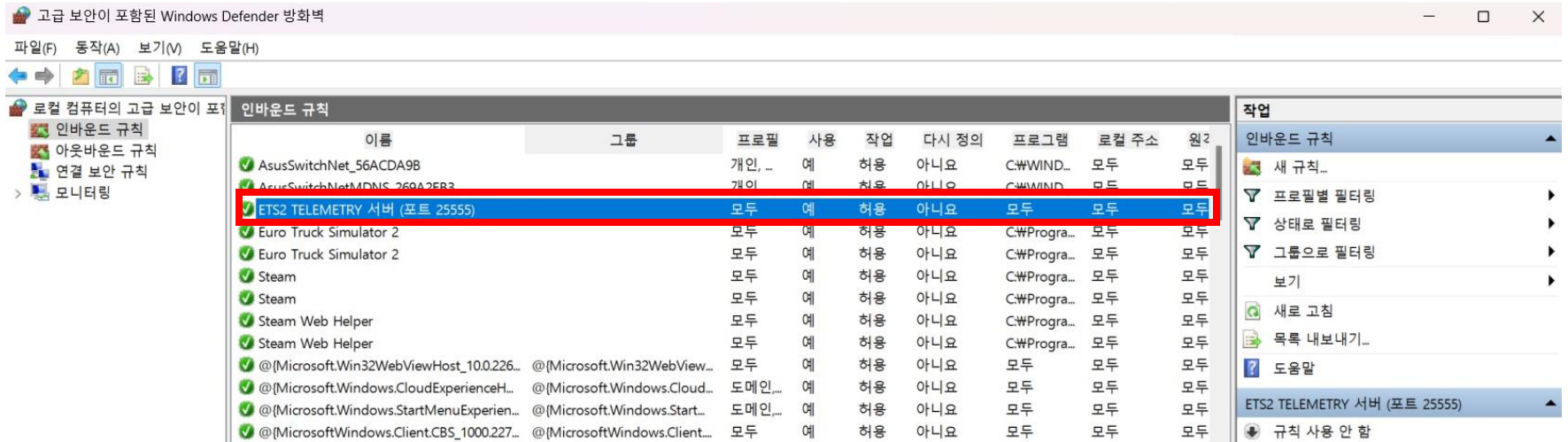
- 차량 상태 정보 : 모두 CAN 통신으로 수신
 - 실차 환경 구성 제약으로 시뮬레이터 활용
- 시뮬레이터
 - 유로 트럭 시뮬레이터 2
- 차량 정보 확보를 위한 Telemetry App
 - 시뮬레이터의 차량 상태 정보를 실시간으로 확보
 - <https://github.com/icaros7/ets2-kor-telemetry-server>
- Server 설정으로 Rest API 제공

차량 시뮬레이터 설정

Ets2Telemetry	2025-08-12 오후 2:34	텍스트 문서	2KB
log4net.dll	2025-08-12 오후 2:33	응용 프로그램 확장	294KB
preview	2025-08-12 오후 2:33	PNG 파일	30KB
System.Net.Http.Formatting.dll	2025-08-12 오후 2:33	응용 프로그램 확장	181KB
System.Web.Cors.dll	2025-08-12 오후 2:33	응용 프로그램 확장	26KB
System.Web.Http.dll	2025-08-12 오후 2:33	응용 프로그램 확장	461KB
System.Web.Http.Owin.dll	2025-08-12 오후 2:33	응용 프로그램 확장	61KB
 Ets2Telemetry	2025-08-12 오후 2:33	응용 프로그램	468KB
Ets2Telemetry.exe.config	2025-08-12 오후 2:33	CONFIG 파일	4KB

- Telemetry 설치 - [Release 3.2.7.4 · icaros7/ets2-kor-telemetry-server](#)
- 받은 소스 압축 해제
- server -> Ets2Telemetry.exe 실행 -> 설치

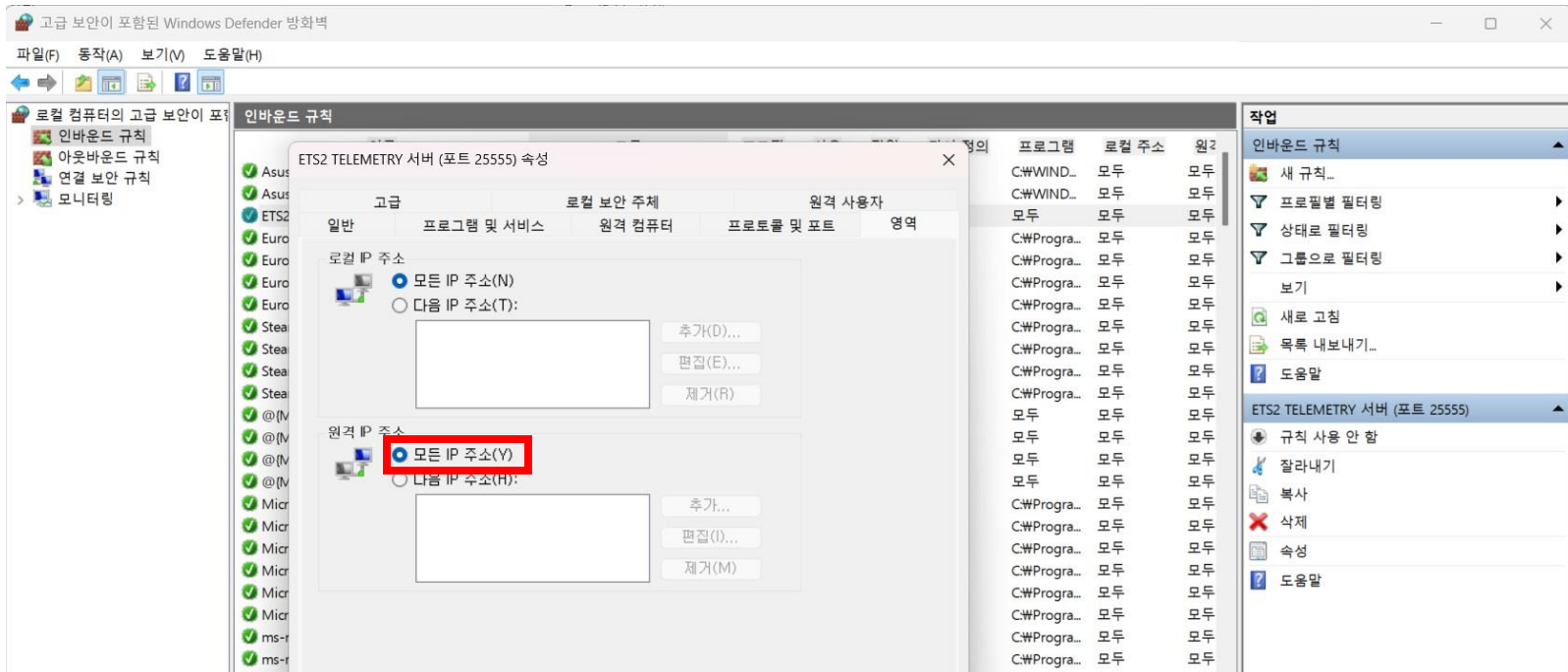
차량 시뮬레이터 설정



• 차량 정보 제공을 위한 방화벽 설정

- 윈도우 검색창 'Windows Defender 방화벽' 검색
- 고급 설정 -> 인바운드 규칙
- ETS2 TELEMETRY 서버 -> 속성

차량 시뮬레이터 설정

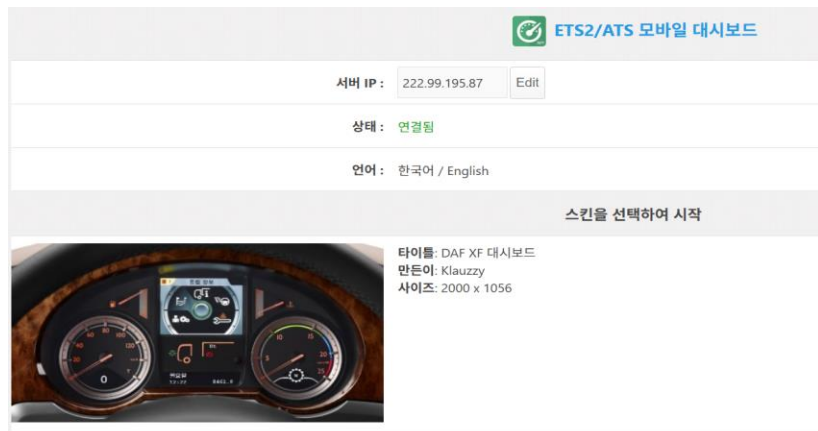


- (필요시) 차량 정보 제공을 위한 방화벽 설정
 - 방화벽 인바운드 설정에서 25555 포트를 전역 네트워크 수신으로 설정

차량 시뮬레이터 설정

- **실행 순서**
 - Telemetry App 실행 후, 시뮬레이터 실행
 - Connected 상태 확인
 - PC의 IP 주소를 사용해 외부에서 접근
- **Web App**
 - `http://${pc-ip4-address}:25555/`
- **Rest API**
 - `http://${pc-ip4-address}:25555/ets2/api/telemetry`

수신 결과



• Web app

- 웹 주소 창에 'http://\${pc-ip4-address}:25555/' 입력
- 원하는 스타일의 대시 보드에 접속해 실시간으로 차량 상태 정보 출력

• Telemetry Information through Rest API

- 웹 주소 창에 'http://\${pc-ip4-address}:25555/ets2/api/telemetry' 입력
- 그림과 같이 웹페이지에 차량 상태 정보 출력

```
{
  "game": {
    "connected": true,
    "gameName": "ETS2",
    "paused": false,
    "time": "0001-01-04T02:17:00Z"
  },
  "id": "daf",
  "make": "DAF",
  "model": "XF",
  "speed": 0.000665952743,
  "cruiseControlSpeed": 0.0,
  "engineRpmMax": 2200.0,
  "fuel": 1248.27466,
  "fuelCapacity": 1400.0,
  "fuelAverageConsumption": 0.186776921,
  "wearWheels": 0.029407274,
  "userSteer": 0.0,
  "userThrottle": 0.0,
  "userBrake": 0.0,
  "retarderBrake": 0,
  "retarderStepCount": 3,
  "parkBrakeOn": true,
  "motorBrakeOn": false,
  "brakePressureWarningOn": false,
  "airPressureWarningValue": 69.6,
  "airPressureEmergencyOn": false,
  "oilTemperature": 16.2492638,
  "waterTemperatureWarningOn": false,
  "waterTemperatureWarningValue": 0.0,
  "headlightOn": true,
  "blinkerLeftActive": false,
  "blinkerRightActive": false,
  "blinkerLeftOn": false,
  "blinkerRightOn": false,
  "beaconOn": false,
  "lightsBrakeOn": false,
  "lightsReverseOn": false,
  "differentialRatio": 0.0,
  "clutchOn": false
}
```

D3-G에서 수신 처리

- D3-G 의 Controller App 에서 REST API 에 접속
- Rest API 응답 (JSON) 데이터 파싱
- Data_parsing.py 예제 코드 참조

```
import json, time, http.client
from urllib.parse import urlparse

URL      = "http://222.99.195.87:25555/api/ets2/telemetry"
POLL_HZ  = 30
TIMEOUT  = 2.0
```

```
u = urlparse(URL)
host = u.hostname
port = u.port or 80
path = (u.path or "/" ) + (("?" + u.query) if u.query else "")
period = 1.0 / POLL_HZ

conn = http.client.HTTPConnection(host, port, timeout=TIMEOUT)
next_t = time.monotonic()

print("[INFO] Start polling:", URL)
print("[INFO] Press Ctrl+C to stop.")

try:
    while True:
        try:
            conn.request("GET", path, headers={
                "User-Agent": "ETS2-Client/1.0",
                "Connection": "keep-alive",
                "Cache-Control": "no-cache",
                "Pragma": "no-cache",
            })
```

- **모듈 импорт**
 - json, time, http.client, urllib.parse 모듈 불러오기
- **설정 값 정의**
 - URL : Telemetry REST API 주소
 - POLL_HZ : 데이터 요청 주기(30Hz)
 - TIMEOUT : 요청 타임아웃(2초)
- **URL 파싱 및 연결 설정**
 - urlparse()로 host, port, path 추출
 - HTTPConnection()으로 서버와 연결 생성
- **Polling 시작**
 - [INFO] 메시지 출력 후 무한 루프 시작
 - GET 요청으로 Telemetry 데이터 수신
 - 응답은 JSON 형태로 처리 예정

D3-G에서 수신 처리

- D3-G 의 Controller App 에서 REST API 에 접속
- Rest API 응답 (JSON) 데이터 파싱
- Data_parsing.py 예제 코드 참조

```
def _scale_payload(js):
    rpm = _get_num(js, "truck.engineRpm")
    speed = _get_num(js, "truck.speed")
    fuel = _get_num(js, "truck.fuel")
    steer = _get_num(js, "truck.userSteer")

    speed_kmh = int(speed) if isinstance(speed, (int, float)) else 0.0

    return {
        "rpm": int(rpm) if isinstance(rpm, (int, float)) else 0,
        "speed_kmh": round(speed_kmh, 2),
        "fuel_l": float(fuel) if isinstance(fuel, (int, float)) else 0.0,
        "steering": float(steer) if isinstance(steer, (int, float)) else 0.0,
    }
```

```
resp = conn.getresponse()
if resp.status == 200:
    data = resp.read()
    js = json.loads(data.decode("utf-8", "ignore"))
    p = _scale_payload(js)
    print(f"rpm={p['rpm']:5d} | speed_kmh={p['speed_kmh']:6.2f} | fuel_l={p['fuel_l']:7.2f}")
else:
    print(f"[WARN] HTTP {resp.status} {resp.reason}")
    try: conn.close()
    except: pass
    conn = http.client.HTTPConnection(host, port, timeout=TIMEOUT)
```

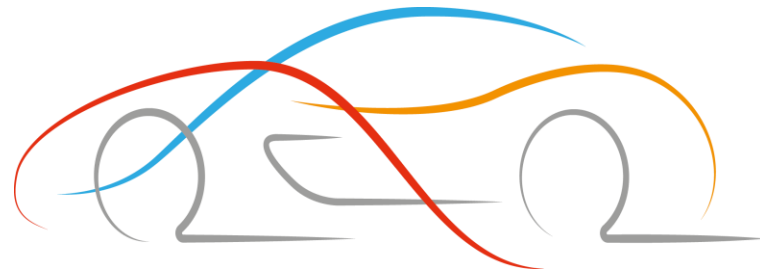
- **_scale_payload(js)**
 - JSON에서 다음 4가지 값을 추출
 - truck.engineRpm → rpm
 - truck.speed → speed
 - truck.fuel → fuel
 - truck.userSteer → steer
 - 딕셔너리 형태로 각 항목 값과 단위 변환 결과 저장
- **응답 처리 (resp = conn.getresponse())**
 - HTTP 상태 코드가 200(정상)일 경우:
 1. 응답 데이터를 읽어 UTF-8로 디코딩
 2. JSON으로 파싱 후 _scale_payload() 호출
 3. 변환된 데이터(rpm, speed_kmh, fuel_l)후 출력
 - 오류 발생 시 HTTP 상태와 사유를 경고로 출력

D3-G에서 수신 결과

- D3-G 에서 예제 코드 실행
 - `$ python3 Data_parsing.py`

```
Polling http://222.99.195.87:25555/api/ets2/telemetry ... (every 0.5s)
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
RPM=552.8 | Speed=0 km/h | Fuel=1247.5 L | Steering=0.000
```

- 위와 같이 30fps로 값을 받아 출력
- RPM, 속도, 연료량, 휠 상태 외에도 여러 값을 수신해 활용이 가능



Thank You